

SIH26153

NetWatch — How to Run

AI-Based Network Attack Forecasting from Network Traffic Data
Counterfactual Cyber World Model · SIH-26153 · Problem Owner:
NTRO

A step-by-step guide to setting up the environment, running the dashboard, ingesting traffic data and using the forecasting pipeline.

1. Overview

NetWatch is an offline, AI-based Network Attack Forecasting system. It ingests network traffic (PCAP, PCAPNG, CSV flow records or JSONL events, optionally wrapped in a ZIP archive), builds per-window network states, and uses a trained LSTM World Model plus an ensemble of detection engines to forecast the next attack stages. A counterfactual simulator evaluates defensive actions and recommends the one that minimises future risk. Everything is served through a Flask dashboard and runs fully offline — no cloud or external inference.

2. Prerequisites

- Python 3.10 or newer
- pip (comes with Python)
- A GitHub account / Git CLI only if you clone the repo
- Internet access only at setup time (installing packages); runtime is offline

3. Project Setup

3.1 Clone the repository

Clone the repository (or copy the project folder onto your machine).

```
git clone https://github.com/sihcodesprit/external_hackathon.git cd
external_hackathon
```

3.2 Create a virtual environment

Windows (PowerShell / Command Prompt):

```
python -m venv venv venv\Scripts\activate
```

macOS / Linux:

```
python3 -m venv venv source venv/bin/activate
```

3.3 Install dependencies

```
pip install -r requirements.txt
```

The requirements include Flask, NumPy, scikit-learn, PyTorch, PyYAML, SHAP and pytest. A CPU-only PyTorch build is sufficient.

3.4 Optional environment file

Copy the template and edit as needed (the .env file is gitignored):

```
copy .env.example .env # Windows cp .env.example .env # macOS / Linux
```

4. Obtain Pre-trained Models (recommended)

The repository ships with placeholder JSON linear checkpoints (NaN weights). Training real LSTM weights is done in Colab — one time — and produces a model ZIP that you upload to the dashboard.

4.1 Train in Google Colab

- Open **NetWatch_Colab.ipynb** in the repository on Google Colab.
- Run all cells. The notebook trains the LSTM World Model, the ensemble baselines, the feature scaler and supporting artifacts.
- It downloads a ZIP named **netwatch_trained_models.zip**.

4.2 Upload the ZIP to the dashboard

After the dashboard is running (Section 5), open the **Models** page (/models) and drop the **netwatch_trained_models.zip** file. The .pt / .pkl / .json / .npz artifacts inside are extracted into data/data/models and the pipeline reloads with real weights.

5. Run the Dashboard

While the virtual environment is active, from the project root:

```
python run.py
```

The server starts on **http://localhost:5000** (open it in your browser). The pipeline loads lazily on the first request; dashboard pages train/load models as needed and remain fully offline.

5.1 CLI options

Command	What it does
python run.py	Start the dashboard (default port 5000)
python run.py --port 8080	Start the dashboard on a custom port
python run.py --no-pipeline	Skip pre-running the pipeline; serve pages only
python run.py --pipeline-only	Run the pipeline end-to-end and print a JSON report, then exit

6. Ingest Network Traffic

From the **Upload** page you can supply your own capture data. Supported formats:

Type	Description	Example
PCAP / PCAPNG	Raw packet captures (requires optional scapy to enable)	capture.pcapng
CSV	Flow-record/feature tables	flows.csv
JSONL	JSON events, one record per line	events.jsonl
ZIP	Any archive wrapping one of the above; the traffic file is auto-detected (pcapng > pcap > csv > jsonl). A ZIP with only model artifacts is treated as a model bundle.	capture_bundle.zip

After ingestion the dashboard rebuilds network states, runs the forecast + counterfactual simulation, and every page reflects the new session data.

7. Dashboard Pages

Page	Route	Content
Dashboard	/dashboard	Current risk, stage, recommendation and forecast summary
Topology	/topology	Dynamic network topology of the ingested session
Forecast Radar	/radar	Future risk timeline
Attack Graph	/graph	Predictive attack graph (current + predicted stages)
Counterfactual	/counterfactual	What-if defence simulation and recommended action
MITRE Stages	/stages	Attack stage timeline + MITRE trajectory
Explainability	/explainability	SHAP / top contributing features
Ensemble	/ensemble	Multi-engine detection consensus scores
Models	/models	Install model ZIPs, view active artifacts, reload
Evaluation	/evaluation	World Model vs baseline metrics
Test Center	/test-center	Model validation checks
Scenarios	/scenarios	Ad-hoc scenario / data explorer
Upload	/upload	Ingest PCAP / CSV / JSONL / ZIP traffic or model bundles

8. Run Tests

From the project root, with the virtual environment active:

```
python -m pytest tests/ -q
```

Expected baseline: **62 passed, 7 failed**. The 7 failures are pre-existing legacy tests ('List' import in pipeline.py, synthetic load_data generator removed, missing-world-model path) and are unrelated to the current workflow.

9. Troubleshooting

Symptom	Solution
'Python' is not recognized	Ensure Python 3.10+ is installed and on PATH; or use 'py -3' on Windows.
Port 5000 already in use	Run 'python run.py --port 8080' and open http://localhost:8080.
ModuleNotFoundError when running run.py	Set PYTHONPATH to the project root, e.g. \$env:PYTHONPATH = ""; then run it.
Forecast shows placeholder / zero risk	Train and upload netwatch_trained_models.zip via the Models page (Section 4).
Uploaded ZIP not detected	ZIP must contain a .pcapng/.pcap/.csv/.jsonl member to be treated as traffic.
Text truncation / encoding issues on Windows	Apply 'git config core.autocrlf true' and re-clone, or run with UTF-8 console.

